

Actua como experto en desarrollo web, y tecnologia, requiero que estudies detalladamente estas dos tecnologias: "https://herd.laravel.com/windows", y "https://filamentphp.com/docs/5.x/introduction/optimizing-local-development#optimizing-local-development", que las entiendas a profundidad, y me expliques detalladamente la forma correcta de usarlas e instalarlas, si estoy en ubuntu, por favor

Voy a revisar la documentación oficial de ambas tecnologías y aterrizarla específicamente a Ubuntu, porque Herd que me pasas apunta a Windows y eso cambia bastante la respuesta.

Ya confirmé el punto clave: Herd no tiene versión para Linux, así que en Ubuntu no se instala como tal. Ahora voy a traducir la intención real de la documentación a una alternativa correcta para Ubuntu y conectarla con Filament.

Quick answer



Actua como experto en desarrollo web, y tecnologia, e informatica requiero que estudies detalladamente estas dos tecnologias: "https://herd.laravel.com/windows", y "https://filamentphp.com/docs/5.x/introduction/optimizing-local-development#optimizing-local-development", que las entiendas a profundidad, y me expliques detalladamente la forma correcta de usarlas e instalarlas, si estoy en ubuntu, por favor

Estoy contrastando la documentación oficial de Herd y Filament con el escenario real de Ubuntu para darte una guía correcta, no una adaptación improvisada de Windows.

Ya tengo la base documental: Herd es oficial para Windows/macOS, mientras que Filament sí aplica perfecto en Ubuntu siempre que montes bien tu stack Laravel/PHP. Me falta cerrar la parte más importante: cuál es el camino correcto en Ubuntu para reemplazar lo que Herd hace por ti.

Pensado durante 3m 9s >

Sí. Ya revisé ambas documentaciones y, aterrizándolo a **Ubuntu**, la conclusión importante es esta: **Herd no es la ruta correcta en Linux**. En la documentación oficial que corresponde a este tema, Herd está documentado como entorno nativo para **Windows** y también tiene documentación para **macOS**; además, su propuesta es darte PHP, nginx, Node.js, dominios *.test, CLI integrada y, en Pro, gestión de servicios como MySQL, PostgreSQL, Redis, Meilisearch y MinIO. Para Ubuntu, la forma correcta de replicar esa experiencia es usar **Laravel Sail** o una instalación nativa de Laravel + Filament. Laravel Herd +3

Filament 5, por su parte, requiere **PHP 8.2+**, **Laravel 11.28+** y **Tailwind CSS 4.1+**. Su instalación oficial para el panel builder es `composer require filament/filament:"^5.0"` seguido de `php artisan filament:install --panels`, y luego puedes crear un usuario con `php artisan make:filament-user`. La documentación también indica que el panel queda accesible en `/admin`. filamentphp.co... +3

Lo que te recomiendo en Ubuntu

Si quieres algo más parecido a Herd, usa:

Ubuntu + Docker Engine + Laravel Sail + Filament

Si quieres algo más simple y liviano, usa:

Ubuntu + instalación nativa de Laravel + Filament + SQLite

La opción más parecida a Herd es **Sail**, porque Laravel lo describe como su entorno Docker por defecto, compatible con **macOS, Linux y Windows**, y porque te permite levantar la app y sus servicios de forma aislada. laravel.com +1

Opción 1: la forma más correcta en Ubuntu si buscas algo "tipo Herd"

Ubuntu + Docker Engine + Laravel Sail + Filament

1) Instala Docker en Ubuntu

Docker documenta dos caminos en Ubuntu: **Docker Desktop** o **Docker Engine**. Para desarrollo web en Ubuntu, yo te recomendaría **Docker Engine** si quieres algo más limpio y más "Linux puro"; Docker dice que Docker Engine en Ubuntu se instala desde su repositorio APT oficial, y también señala que Docker Desktop para Linux es la vía más rápida, aunque tiene consideraciones de licencia para empresas grandes. Docker Docum... +2

Comandos oficiales del repositorio APT de Docker para Ubuntu: Docker Docum...

```
sudo apt update
sudo apt install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyri
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin do
```

Luego valida Docker así: Docker Docum...

 Bash



Docker también advierte que, por defecto, quizás debas usar `sudo` hasta hacer el post-install de Linux para permitir uso sin root. Docker Docum...

2) Instala PHP, Composer y el instalador de Laravel en Ubuntu

Laravel ofrece para Linux el instalador `php.new`, que instala **PHP, Composer y el Laravel installer**. Además, Laravel aclara que también necesitas **Node/NPM o Bun** para compilar assets frontend. [laravel.com](#) +1

</> Bash



Después reinicia la terminal. Eso lo indica la propia guía. [laravel.com](#) +1

3) Crea tu proyecto Laravel

Laravel recomienda crear la aplicación así: [laravel.com](#)

```
laravel new panel-app
```

</> Bash



Laravel también indica que, por defecto, una app nueva queda lista con **SQLite** en `.env`, crea `database/database.sqlite` y ejecuta las migraciones necesarias al momento de crearla. [laravel.com](#) +1

4) Activa Sail en el proyecto

En una aplicación existente, la guía oficial de Sail indica instalarlo así: [laravel.com](#) +1

```
composer require laravel/sail --dev
php artisan sail:install
./vendor/bin/sail up
```



Si no quieres escribir siempre `./vendor/bin/sail`, Laravel recomienda este alias para `bash` o `zsh`: [laravel.com](#)

</> Bash



Agrégallo a `~/.bashrc` o `~/.zshrc` y reinicia la shell. Luego podrás usar simplemente:

</> Bash



Laravel también indica que, una vez levantados los contenedores, la aplicación queda en `http://localhost`. Si usas **Docker Desktop for Linux**, la guía pide usar el contexto `default` con `docker context use default`. [laravel.com](#) +1

5) Instala Filament dentro del flujo de Sail

Laravel aclara que, cuando usas Sail, los comandos que normalmente correrías como `php`, `composer` o `npm` debes ejecutarlos a través de **Sail**. [laravel.com](#) +1

Entonces, para Filament en Ubuntu con Sail, hazlo así:

```
sail composer require filament/filament:"^5.0"  
sail artisan filament:install --panels  
sail artisan make:filament-user
```



Con eso, el panel de Filament queda en:

</> Plain text



porque Sail sirve la app en `http://localhost` y Filament expone el panel en `/admin`.

[laravel.com](#) +1

6) Cómo trabajar día a día con Sail + Filament

Tus comandos habituales quedarían así: [laravel.com](#) +1

```
sail up -d  
sail artisan migrate
```

```
sail artisan make:filament-resource Post
sail artisan make:filament-user
sail stop
```

Y para frontend, siguiendo la lógica de Sail, también ejecutarías npm dentro del entorno:

```
sail npm install
```

</> Bash



Eso es lo más cercano, en Ubuntu, a la experiencia "todo integrado" de Herd. [laravel.com](#) +1

Opción 2: la forma más simple en Ubuntu si no quieres Docker

Ubuntu + Laravel nativo + Filament

Esta opción no imita Herd, pero es muy práctica para aprender o para proyectos pequeños.

Laravel documenta el flujo con `php.new`, creación de proyecto con `laravel new`, y arranque con `composer run dev`; además indica que la app queda en

`http://localhost:8000`. [laravel.com](#) +2

El flujo sería:

</> Bash



Luego:

```
laravel new panel-app
cd panel-app
composer require filament/filament:"^5.0"
php artisan filament:install --panels
php artisan make:filament-user
npm install && npm run build
composer run dev
```

Después abres:

</> Plain text



Ese flujo mezcla Laravel Installation + Filament Installation de forma coherente para Ubuntu. Laravel pide Node/NPM o Bun para assets, y Filament usa `npm` en su proceso de assets/scaffolding. [laravel.com](#) +2

Cómo entender Herd correctamente, aunque estés en Ubuntu

Herd no es "Filament". Herd es el **entorno local**. Filament es el **framework/admin panel** que corre dentro de Laravel. Herd te resuelve la parte de:

- PHP
- nginx
- Node.js
- dominios `*.test`
- herramientas CLI
- servicios opcionales tipo DB/Redis/Meilisearch en Pro

Filament, en cambio, se instala **dentro** de tu proyecto Laravel y te crea el panel administrativo. En Windows/macOS podrías usar Herd + Filament. En Ubuntu, lo correcto es sustituir Herd por **Sail** o por un stack nativo. [Laravel He...](#) +3

Qué partes del documento de Filament "Optimizing local development" sí aplican en Ubuntu

Esa guía de Filament sí es muy útil en Ubuntu. Recomendando:

- revisar si **OPcache** está habilitado con


```
php -r "echo 'opcache.enable => ' . ini_get('opcache.enable') . PHP_EOL;"
```
- si no está activo, habilitar `opcache.enable=1`
- ajustar, si hace falta, `opcache.memory_consumption=128` y


```
opcache.max_accelerated_files=10000
```
- desactivar Debugbar con `DEBUGBAR_ENABLED=false`
- desactivar Xdebug con `xdebug.mode=off`
- cachear iconos con `php artisan icons:cache` [filamentphp.co...](#)

Lo único de esa guía que **no te aplica en Ubuntu** es la parte específica de “**Disabling view debugging in Laravel Herd**”, porque ese apartado habla explícitamente del debugging visual de Herd en **macOS y Windows**. filamentphp.co... +2

Mi recomendación final

Para Ubuntu, yo te diría:

- Si quieres la experiencia más parecida a Herd: usa **Docker Engine + Sail + Filament**.
- Si quieres empezar rápido y simple: usa **php.new + Laravel nativo + Filament + SQLite**.
- No intentaría forzar Herd en Ubuntu como si fuera una instalación oficial equivalente, porque la documentación revisada de Herd para este caso está enfocada en **Windows/macOS**, mientras que Laravel sí te da una ruta oficial clara para Linux con **php.new** y con **Sail**. Laravel He... +3

Puedo dejarte ahora una guía exacta, paso a paso, pensada para tu Ubuntu específico, por ejemplo **Ubuntu 22.04 o 24.04**, con todos los comandos en orden para que te quede instalado **Laravel + Sail + Filament** sin errores.

     ...  Fuentes

Bueno presta atencion, esto es lo que requiero, necesito desarrollar un proyecto, cuyo proposito es ser una agenda estudiantil, para una seccion en especifico de un grado academico de bachillerato Venezolano, en la cual estos estudiantes tenga la oportunidad de iniciar sesion en esta plataforma, para principalmente ver su horario de estudiantil, y tambien que puedan ver las tareas asignadas a la seccion por materia, es decir, que el profesor de matematicas, de ingles, de castellano, ect, pueda subir la tarea a la pagina, y esta subida, pueda llegar como notificacion a todos los estudiantes relacionados a dicha seccion, y que estos tengas su numero de tlf relacionado, esta ultima parte la podemos desarrollar despues de la primera parte, necesito que me guies paso por paso y de forma detallada, el como debo empezar a desarrollar este

proyecto, considerando que estoy en windows 11 con WSL2 y desarrollo en WSL2, segun tu respuesta anterior esta: "Si quieres algo más simple y liviano, usa:

Ubuntu + instalación nativa de Laravel + Filament + SQLite", es la forma mas simple de desarrollar considerando las tecnologicas estudiadas en este chat, guíame en primera instancias como desarrollar en forma local primero, por favor, si entiendes y puedes hacer todo lo que te pido?

Ya estoy aterrizando esto al proyecto real, no solo a la instalación. Voy a estructurarte el arranque local en WSL2 con una ruta de MVP clara: estudiantes, profesores, secciones, horario y tareas primero; notificaciones telefónicas después.

Ya cerré la decisión de arquitectura inicial: para este proyecto no conviene usar Filament para todo desde el día 1. Lo más limpio es usar **Laravel para el portal estudiantil** y **Filament como panel administrativo para docentes/admin**, porque así arrancas rápido sin enredar la experiencia del estudiante.

Pensado durante 2m 49s >

Sí, entiendo exactamente lo que quieres construir.

Tu **MVP local** debería ser esto:

1. **Estudiantes** inician sesión.
2. Ven su **horario** de clases.
3. Ven las **tareas** publicadas para su sección, separadas por materia.
4. **Docentes y administrador** cargan y gestionan materias, horarios y tareas desde un panel interno.
5. Más adelante agregamos **notificaciones** dentro de la plataforma y luego notificaciones al teléfono.

Para arrancar en **Windows 11 + WSL2 + Ubuntu**, la forma más simple y ordenada es esta:

- **Laravel 12** como base del proyecto.
- **Starter kit Livewire** para resolver login/registro sin meterte de lleno en frontend complejo.

- **SQLite** para desarrollo local.
- **Filament 5** para el panel de administración de docentes/admin.
- **Blade/Livewire** para la vista de estudiantes.

Laravel hoy documenta instalación en Linux con `php.new`, creación del proyecto con `laravel new`, y sus starter kits ya cubren autenticación; además, Laravel 12 incluye starter kits nuevos para React, Vue, Svelte y **Livewire**. Filament 5 requiere **PHP 8.2+**, **Laravel 11.28+** y **Tailwind 4.1+**, se instala con `composer require filament/filament:"^5.0"` y `php artisan filament:install --panels`, y deja el panel en `/admin`. Filament además puede servir para paneles, portales y aplicaciones con múltiples paneles. [laravel.com](#) +4

Decisión técnica inicial

No te recomiendo empezar intentando que **todo** sea Filament.

La ruta más limpia para tu caso es:

- **Filament** para el lado interno: administrador y docentes.
- **Laravel normal** para el lado del estudiante: login, dashboard, horario y tareas.

Eso te evita mezclar desde el día 1 la experiencia administrativa con la experiencia estudiantil. Para controlar qué puede hacer cada usuario, Laravel recomienda trabajar con **gates** y **policies** cuando la autorización depende de acciones o modelos concretos.

[laravel.com](#) +1

Fase 0: prepara bien el entorno en WSL2

Antes de crear nada, guarda el proyecto **dentro del sistema de archivos Linux de WSL**, no en `/mnt/c/...`. Microsoft recomienda eso para mejor rendimiento cuando trabajas desde Ubuntu/WSL; el patrón recomendado es algo como `/home/tu_usuario/proyecto`.

[Microsoft Learn](#) +1

Usa algo así:

```
mkdir -p ~/proyectos
```

</> Bash



Fase 1: instala Laravel en Ubuntu (WSL2)

Laravel documenta que en Linux puedes instalar **PHP, Composer y el Laravel installer** con:

</> Bash



Después debes **cerrar y abrir** la terminal. Laravel también indica que además de PHP y Composer necesitas **Node y NPM o Bun** para compilar assets del frontend. laravel.com

Para Node en WSL, una opción muy cómoda es **nvm**, cuyo repositorio oficial indica que funciona en shells POSIX y específicamente en **Windows WSL**. La instalación oficial es esta:

</> Bash



Luego cierras y abres la terminal, verificas `nvm`, e instalas una versión LTS de Node:

```
command -v nvm
nvm install --lts
node -v
npm -v
```

`nvm` está pensado justo para manejar versiones de Node por shell/usuario, y en WSL encaja muy bien. [GitHub](#) +2

También instala SQLite en Ubuntu para el entorno local:

```
sudo apt update
```

</> Bash



Fase 2: crea el proyecto base

Laravel documenta que, una vez instalado el entorno, creas la app con:

</> Bash



y que el instalador te va guiando para escoger base de datos, testing framework y starter kit. Luego, al entrar al proyecto, puedes levantar el entorno con `composer run dev`, que arranca el servidor local, el worker de colas y Vite. laravel.com

Haz esto:

```
cd ~/proyectos
laravel new agenda-estudiantil
cd agenda-estudiantil
```

Cuando el instalador te pregunte, te recomiendo elegir:

- **Starter kit:** Livewire
- **Database:** SQLite
- **Authentication:** la opción normal integrada
- **Testing:** Pest o PHPUnit

Yo usaría Pest solo por comodidad, pero cualquiera sirve.

La razón de elegir **Livewire** es que Laravel 12 ya lo contempla como starter kit oficial y esos starter kits traen autenticación integrada: login, registro, recuperación de contraseña y verificación de correo. laravel.com +2

Después:

```
npm install
npm run build
php artisan migrate
composer run dev
```

Si todo está bien, abres el proyecto local en el navegador. Laravel indica que ese flujo queda listo con el script `dev`. laravel.com

Fase 3: instala Filament para docentes y administrador

Dentro del proyecto:

```
composer require filament/filament:"^5.0"  
php artisan filament:install --panels  
php artisan make:filament-user
```

Filament documenta que eso crea el provider del panel, normalmente `app/Providers/Filament/AdminPanelProvider.php`, y que luego puedes entrar por `/admin`. [filamentphp.co...](https://filamentphp.com/docs/3.x/panels/installation)

Entonces tu app local quedará, en esencia, así:

- **Portal estudiante:** rutas normales de Laravel
- **Panel docente/admin:** `/admin`

Fase 4: define bien el alcance del MVP

No empieces por notificaciones telefónicas.

Primero cierra esto:

- usuarios
- roles
- secciones
- materias
- horario
- tareas

La primera versión debe responder esta pregunta:

"Un estudiante de una sección concreta puede entrar y ver su horario y las tareas activas de su sección?"

Si eso funciona, el proyecto ya tiene valor real.

Fase 5: modelo de datos recomendado para empezar

Yo te recomiendo este diseño inicial, simple pero correcto:

Tabla users

Una sola tabla para todos los usuarios.

Campos clave:

- id
- name
- email
- password
- role → admin, teacher, student

Tabla academic_sections

Representa la sección académica.

Campos:

- id
- name → ejemplo: 5to Año "A"
- school_year → ejemplo: 2026-2027

Tabla subjects

Representa materias.

Campos:

- id
- name → Matemáticas, Inglés, Castellano
- code nullable

Tabla student_profiles

Datos extra del estudiante.

Campos:

- id
- user_id

- academic_section_id
- phone nullable

Tabla teacher_profiles

Datos extra del docente.

Campos:

- id
- user_id
- phone nullable

Tabla section_subject_teachers

Relación entre sección, materia y docente.

Campos:

- id
- academic_section_id
- subject_id
- teacher_id → referencia a users

Tabla schedules

Horario por sección.

Campos:

- id
- academic_section_id
- subject_id
- teacher_id nullable
- weekday → 1 a 5
- start_time
- end_time

Tabla assignments

Tareas.

Campos:

- id
- academic_section_id
- subject_id
- teacher_id
- title
- description
- due_date nullable
- published_at
- is_active

Con eso cubres perfectamente el MVP.

Fase 6: crea las entidades una por una

En vez de crear todo a mano mezclado, crea primero modelos y migraciones.

```
php artisan make:model AcademicSection -m  
php artisan make:model Subject -m  
php artisan make:model StudentProfile -m  
php artisan make:model TeacherProfile -m  
php artisan make:model SectionSubjectTeacher -m  
php artisan make:model Schedule -m  
php artisan make:model Assignment -m
```

Después editas las migraciones y agregas las relaciones.

Tu prioridad al principio debe ser:

1. modificar users para agregar role
2. crear academic_sections
3. crear subjects
4. crear student_profiles
5. crear teacher_profiles
6. crear schedules
7. crear assignments

Luego:

 **Bash**



```
php artisan migrate
```

Fase 7: implementa autenticación y roles simples

Como Laravel ya te deja lista la base de autenticación con el starter kit, lo primero que haría es usar la misma tabla `users` y manejar el rol con un campo `role`. Laravel documenta que sus starter kits ya integran rutas y lógica de login, registro, recuperación y verificación de correo. [laravel.com](#) +1

Para el MVP:

- `admin` entra a `/admin`
- `teacher` entra a `/admin`
- `student` entra al portal normal

No metas aún paquetes de permisos avanzados.

Primero hazlo funcionar con una lógica simple y clara.

Fase 8: usa Filament para lo administrativo

Filament Resources te permiten generar CRUD rápidamente. Filament documenta `php artisan make:filament-resource NombreModelo`, y eso te crea recursos con páginas de listar, crear y editar dentro de `app/Filament/Resources`. [filamentphp.co...](#)

Crea estos recursos en Filament:

 **Bash**



```
php artisan make:filament-resource AcademicSection
php artisan make:filament-resource Subject
php artisan make:filament-resource Schedule
php artisan make:filament-resource Assignment
php artisan make:filament-resource User
```

Luego ajustas el `UserResource` para que:

- admin gestione docentes y estudiantes
- docentes no puedan ver o editar todo indiscriminadamente

Con Filament puedes además personalizar etiquetas y navegación del panel. [filamentphp.co...](https://filamentphp.com/docs/3.x/panels/navigation)

Fase 9: construye primero el portal del estudiante

No empieces por dashboards complejos.

Empieza por estas rutas:

 PHP



```
/dashboard  
/horario  
/tareas  
/tareas/{assignment}
```

Página /dashboard

Debe mostrar:

- nombre del estudiante
- sección a la que pertenece
- tareas recientes
- próximas entregas

Página /horario

Debe mostrar:

- tabla semanal
- bloques por día
- materia y horario

Página /tareas

Debe mostrar:

- lista de tareas activas
- materia
- fecha de entrega
- detalle

El estudiante solo debe ver datos de **su sección**, nunca de otra.

Fase 10: controla permisos desde el inicio

Laravel recomienda **gates** y **policies** para autorización. En tu caso, te recomiendo policies para modelos como `Assignment` y `Schedule`. laravel.com

Reglas mínimas:

- **student**: solo puede ver horario y tareas de su sección.
- **teacher**: solo puede crear/editar tareas de las materias/secciones que tiene asignadas.
- **admin**: puede ver y gestionar todo.

Aunque al principio lo resuelvas simple, deja esa idea clara desde ya.

Fase 11: crea datos de prueba

Tu proyecto se vuelve mucho más fácil de desarrollar si creas un seeder inicial.

Deberías sembrar:

- 1 admin
- 2 docentes
- 1 sección
- 5 materias
- 10 estudiantes
- horario básico semanal
- 3 tareas de ejemplo

Eso te permitirá probar todo sin estar creando registros manualmente a cada rato.

Fase 12: deja las notificaciones para la segunda iteración

Para la siguiente etapa, Laravel ya trae una base sólida para notificaciones. La documentación oficial indica que puede enviar notificaciones por **email**, **SMS**, **Slack** y también guardarlas en **base de datos** para mostrarlas dentro de la interfaz; además, el

broadcasting puede llevar eventos del backend al frontend por WebSockets mediante **Laravel Reverb, Pusher o Ably**. [laravel.com](#) +1

Por eso, cuando lleguemos a esa fase, el orden correcto sería:

1. **notificación interna en la web** al estudiante
2. **correo electrónico**
3. **SMS / WhatsApp** según el proveedor que escojas

No comenzaría por teléfono desde el día 1.

Orden exacto en que yo lo desarrollaría

Sprint 1

Montar entorno y proyecto:

- WSL2
- Laravel
- Livewire starter kit
- SQLite
- Filament

Sprint 2

Modelo de datos:

- users con role
- sections
- subjects
- student_profiles
- teacher_profiles
- schedules
- assignments

Sprint 3

Panel admin/docente con Filament:

- CRUD de secciones
- CRUD de materias

- CRUD de horarios
- CRUD de tareas
- sección básica de usuarios

≡ ChatGPT 5.4 Thinking ▾



Portal estudiante:

- login
- dashboard
- horario
- tareas

Sprint 5

Permisos:

- students ven solo lo suyo
- teachers publican solo en su sección/materia
- admin controla todo

Sprint 6

Notificaciones internas

Ese es el orden más sano.

Comandos mínimos que deberías ejecutar primero

Te los dejo compactos:

⌗ Bash



1) Trabaja dentro de Linux, no en /mnt/c

`mkdir -p ~/proyectos`

`cd ~/proyectos`

2) Instala Laravel base

`/bin/bash -c "$(curl -fsSL https://php.new/install/linux/8.4)"`

3) Cierra y abre terminal

4) Instala nvm y Node LTS

`curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh | bas`

```
# cierra y abre terminal
command -v nvm
nvm install --lts
node -v
npm -v

# 5) Instala SQLite
sudo apt update
sudo apt install -y sqlite3

# 6) Crea proyecto
laravel new agenda-estudiantil
cd agenda-estudiantil

# 7) Instala dependencias frontend y migra
npm install
npm run build
php artisan migrate

# 8) Instala Filament
composer require filament/filament:"^5.0"
php artisan filament:install --panels
php artisan make:filament-user

# 9) Levanta entorno local
composer run dev
```

Todo eso está alineado con la documentación actual de Laravel y Filament para Linux/local development. laravel.com +3

Ajustes útiles para que vaya más fluido en local

Filament recomienda algunas optimizaciones locales opcionales:

</> Bash



```
php -r "echo 'opcache.enable => ' . ini_get('opcache.enable') . PHP_EOL;"
php --ini
php artisan icons:cache
```

Y, si llegas a notar lentitud, su guía sugiere revisar OPcache, desactivar Debugbar con:

</> env



```
DEBUGBAR_ENABLED=false
```

y desactivar Xdebug con:

```
</> INI
```



```
xdebug.mode=off
```

En tu caso, además, al trabajar con WSL2, ayuda mucho mantener el proyecto dentro de `/home/...` y no en la ruta montada de Windows. [filamentphp.co... +4](#)

Mi recomendación final

Empieza con esta meta concreta:

"Esta semana voy a tener una app local donde un admin carga una sección, materias, horario y tareas, y un estudiante entra y ve su horario y sus tareas."

Ese objetivo es realista, útil y técnicamente correcto.

En el siguiente mensaje te puedo dejar la **estructura exacta de migraciones y modelos** para este proyecto, ya pensada para copiarla en Laravel.

        Fuentes

Voy por este comando: "curl -o-
https://raw.githubusercontent.com/nvm-
sh/nvm/v0.40.4/install.sh | bash", pero compruebo que
exista node en WSL2, con: "node -v", y recibo como
respuesta: "v22.13.0", instalo estos comandos: "curl -o-
https://raw.githubusercontent.com/nvm-
sh/nvm/v0.40.4/install.sh | bash", "command -v nvm
nvm install --lts
node -v
npm -v"? , que me indicas?

Estoy verificando dos cosas antes de decirte qué ejecutar: si realmente necesitas `nvm` teniendo Node ya instalado, y si esa versión que tienes te sirve bien para Laravel/Filament en local.

Pensado durante 26s >

Sí: como ya tienes `node -v` respondiendo `v22.13.0`, no estás obligado a instalar `nvm` ahora mismo. Para Laravel, la documentación oficial solo exige tener **Node y NPM o Bun** para compilar los assets del frontend; no exige `nvm` como requisito. laravel.com

Lo importante ahora es comprobar también `npm` :

</> Bash



```
npm -v
```

Si `npm -v` responde bien, puedes seguir con el proyecto sin instalar `nvm` todavía. laravel.com

`nvm` te conviene cuando quieres **gestionar varias versiones de Node** desde la terminal. Su README oficial dice justamente que sirve para instalar y usar distintas versiones de Node rápidamente, y que funciona en shells POSIX, incluyendo **Windows WSL**. [GitHub](https://github.com/nvm-sh/nvm) +1

Entonces, mi indicación concreta es esta:

Caso 1: quieres avanzar ya, sin complicarte

Haz esto:

</> Bash



```
node -v
```

```
npm -v
```

Si ambos responden bien, sigue con Laravel. No instales `nvm` por ahora. Laravel solo necesita que Node/NPM estén disponibles para compilar frontend. laravel.com

Caso 2: quieres dejar tu entorno más ordenado desde el inicio

Sí instala `nvm`, pero hazlo sabiendo que su valor real es **administrar versiones**, no “hacer que Node exista”. La instalación oficial es esta: [GitHub](https://github.com/nvm-sh/nvm)

</> Bash



```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh | bas
```

Luego cierra y abre la terminal, o recarga tu shell, y verifica:

</> Bash



```
command -v nvm
```

Después instalas una LTS y la usas:

</> Bash



```
nvm install --lts  
node -v  
npm -v
```

Eso está alineado con el uso oficial de `nvm`, que además deja configurada una versión por defecto cuando instalas la primera versión con él. [GitHub](#)

Lo que yo te recomiendo en tu caso

Como ya tienes Node funcionando, haría esto primero:

</> Bash



```
node -v  
npm -v
```

Si ambos funcionan, sigue con:

</> Bash



```
laravel new agenda-estudiantil
```

y no toques `nvm` todavía. Laravel solo pide Node/NPM o Bun para los assets. [laravel.com](#)

Instalaría `nvm` más adelante solo si pasa alguna de estas cosas:

- quieres cambiar entre varias versiones de Node para distintos proyectos
- algún paquete te da problemas y quieres bajar a una LTS fácilmente

- quieres estandarizar mejor tu entorno de desarrollo en WSL [GitHub](#) +1

Mi respuesta corta aplicada a tu punto exacto es:

- **No, no es obligatorio ejecutar esos comandos de `nvm` ahora mismo.**
- **Sí puedes instalarlos si quieres un entorno más controlado.**
- **Para arrancar el proyecto local, basta con que `node` y `npm` funcionen bien.**

[laravel.com](#) +1

El siguiente paso más sensato es que me pegues la salida de:

```
</> Bash

npm -v
```

y con eso te digo si seguimos directo a crear el proyecto Laravel.

     ...  Fuentes



Voy por esta instrucción: "laravel new agenda-estudiantil
cd agenda-estudiantil", en el capture de pantalla, lo que señalo con el rectángulo en color rojo, para mi caso, cual es la mejor opción, y a que hace referencia?

Pensado durante 48s >

Sí. En **esa pantalla**, para tu caso, la mejor opción es:

Laravel's built-in authentication. [laravel.com](#)

Eso hace referencia a **qué proveedor o sistema de autenticación va a traer preconfigurado tu proyecto** cuando termine de crearse el starter kit. Laravel documenta que, por defecto, los starter kits usan la autenticación integrada de Laravel y con eso obtienes **login, registro, reseteo de contraseña, verificación de correo y más.** laravel.com

Qué significa cada opción

1) Laravel's built-in authentication

Es la opción correcta para tu proyecto ahora mismo.

Te deja una base local y simple para manejar:

- estudiantes
- docentes
- administrador
- login/logout
- recuperación de contraseña
- validaciones básicas

Como tu proyecto es una **agenda estudiantil** y estás construyendo un **MVP local en WSL2**, esta opción es la más sana porque **no depende de un servicio externo** y te deja todo dentro de Laravel desde el inicio. La documentación oficial indica que esta es la opción por defecto de los starter kits. laravel.com

2) WorkOS

Esto es para usar **WorkOS AuthKit** como proveedor externo de autenticación. Laravel indica que esa variante añade:

- inicio de sesión social
- passkeys
- "Magic Auth" por email
- SSO

pero **requiere una cuenta WorkOS** y además configurar variables como

`WORKOS_CLIENT_ID` , `WORKOS_API_KEY` y `WORKOS_REDIRECT_URL` en el `.env` . laravel.com

Para tu caso, **no la elijas ahora.**

Sería útil más adelante solo si algún día quisieras cosas como:

- iniciar sesión con Google o Microsoft

- acceso institucional con SSO
- evitar manejar contraseñas directamente

Para una plataforma escolar local en etapa inicial, eso sería meter complejidad innecesaria.

laravel.com

3) No authentication scaffolding

Eso significa que el instalador **no te monta la base de autenticación**.

O sea: tú tendrías que construir login, registro, recuperación, middleware y flujo de acceso por tu cuenta. La propia etiqueta del instalador lo deja claro al decir “No authentication scaffolding”. laravel.com

Para tu proyecto, **no conviene**.

Mi recomendación concreta en esa pantalla

Selecciona:

Laravel's built-in authentication. laravel.com

Pero aquí hay un detalle importante

En tu captura veo que arriba ya elegiste `React` como starter kit.

Eso **sí funciona**, pero no es lo más simple para el camino que te propuse. Laravel documenta que el starter kit de React usa **Inertia, React 19, TypeScript, Tailwind y shadcn/ui**; en cambio, el starter kit de **Livewire** está pensado como una alternativa más simple, basada en PHP/Blade, para equipos que quieren evitar la complejidad de frameworks SPA como React.

laravel.com

Entonces, para tu proyecto te doy dos caminos:

Camino A — el más simple

Cancelar este proceso y volver a ejecutar `laravel new agenda-estudiantil`, pero esta vez elegir:

- **Starter kit:** Livewire

- **Authentication provider:** Laravel's built-in authentication

Ese es el camino que mejor encaja con lo que te recomendé antes. laravel.com

Camino B — seguir con lo que ya comenzaste

Si ya quieres continuar con React porque te sientes cómodo o quieres una interfaz más moderna desde el inicio, entonces en esa pantalla igual debes elegir:

- **Authentication provider:** Laravel's built-in authentication

También funcionará bien. Laravel documenta ese starter kit como un punto de partida moderno con React sobre Inertia. laravel.com

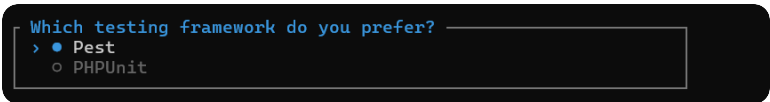
Mi recomendación final para ti

Para tu caso exacto:

- **En esa pantalla:** elige `Laravel's built-in authentication`.
- **Y si quieres la ruta más simple de verdad:** retrocede y crea el proyecto con **Livewire** en vez de React. laravel.com

El siguiente paso, si quieres hacerlo de la forma más correcta, es que me muestres la **pantalla anterior del instalador** o me digas si prefieres que te indique cómo **cancelar este intento y recrearlo con Livewire + autenticación integrada**.

     ...  Fuentes



```
Which testing framework do you prefer?
> • Pest
  ○ PHPUnit
```

Y esto del capture de pantalla, a que hace referencia?

Pensado durante 21s >

Eso hace referencia a **qué framework de pruebas automatizadas quieres que traiga configurado el proyecto desde el inicio**.

En Laravel 12, el framework viene con soporte para **Pest** y **PHPUnit** "out of the box", con `phpunit.xml` ya preparado, carpetas `tests/Feature` y `tests/Unit`, y puedes ejecutar pruebas con `vendor/bin/pest`, `vendor/bin/phpunit` o `php artisan test`. laravel.com

Para tu caso, yo te recomiendo elegir:

Pest

porque para un proyecto nuevo como tu agenda estudiantil suele ser más cómodo de leer y escribir, especialmente cuando estás empezando con Laravel. Laravel además recomienda Pest para proyectos nuevos en pruebas de navegador, lo que refleja que hoy es una opción muy alineada con el ecosistema actual. laravel.com +1

Qué diferencia hay entre ambos

PHPUnit es el framework clásico de pruebas en PHP.

Es muy usado, muy sólido, y gran parte del ecosistema PHP lo conoce.

Pest está construido sobre PHPUnit, pero con una sintaxis más limpia y expresiva.

O sea: si eliges Pest, no estás dejando de usar la base sólida de PHPUnit; simplemente usas una capa más cómoda para escribir tests. Laravel sigue soportando ambos oficialmente.

laravel.com +1

Qué te conviene seleccionar ahora

Para tu proyecto local en WSL2:

- **Authentication provider:** Laravel's built-in authentication
- **Testing framework:** Pest

Esa combinación es una muy buena base para arrancar rápido y sin complejidad innecesaria.

laravel.com

Qué tipo de pruebas harías luego en tu proyecto

Más adelante, con ese proyecto, vas a querer probar cosas como:

- que un estudiante pueda iniciar sesión
- que solo vea tareas de su sección
- que un docente pueda crear una tarea
- que un alumno no pueda entrar al panel admin
- que el horario se muestre correctamente

Eso en Laravel normalmente cae sobre todo en **Feature tests**, que la documentación describe como pruebas que cubren una parte más grande del sistema, incluso una petición HTTP completa; de hecho, Laravel señala que la mayoría de tus pruebas deberían ser de ese tipo. [laravel.com](#)

Entonces, en esa pantalla, selecciona **Pest** y continúa.

El siguiente punto que te puedo guiar es el que viene justo después de terminar el `laravel new`, para validar que el proyecto quedó bien creado antes de instalar Filament.

       Fuentes

```
Do you want to install Laravel Boost to improve AI assisted coding?  
● Yes / ○ No
```

Esta opción, a que hace referencia, y según mi caso cual escojo?

Pensado durante 15s >

Esa opción hace referencia a **si quieres instalar Laravel Boost en el proyecto**.

Laravel Boost es una herramienta oficial para mejorar el trabajo con asistentes de IA dentro de proyectos Laravel. Según la documentación oficial, añade contexto específico de Laravel para agentes de IA, incluyendo guías de código, skills, un servidor MCP y acceso a documentación del ecosistema Laravel ajustada a las versiones que usa tu proyecto. Se instala como dependencia de desarrollo con `composer require laravel/boost --dev` y luego `php artisan boost:install`. [laravel.com](#) +2

Para tu caso, qué debes escoger

Yo te recomiendo:

No

porque ahora mismo tu prioridad es:

- crear el proyecto local
- dejar funcionando autenticación
- montar SQLite
- instalar Filament
- modelar secciones, materias, horarios y tareas

Boost **no es necesario** para que la aplicación funcione. Es una ayuda para desarrollo asistido por IA, no una pieza base de tu agenda estudiantil. Además, Laravel indica que Boost genera archivos de configuración para agentes/editores como `.mcp.json`, `CLAUDE.md`, `AGENTS.md` y `boost.json`, o sea, agrega una capa extra pensada para tooling de IA. [laravel.com](#) +1

Cuándo sí tendría sentido elegir **Yes**

Tendría sentido si desde ya quieres que tu editor o agente de IA trabaje mejor con tu proyecto Laravel, por ejemplo para:

- entender rutas, modelos y base de datos
- consultar documentación Laravel desde el contexto del proyecto
- usar herramientas MCP con agentes compatibles
- mejorar sugerencias de código Laravel específicas del proyecto [laravel.com](#) +1

Mi recomendación práctica

Para este arranque:

- **Laravel Boost:** No
- termina de crear el proyecto
- luego validamos que levante bien
- después instalamos Filament

Más adelante, si quieres, puedes añadir Boost manualmente sin problema con:

 **Bash**



```
composer require laravel/boost --dev
php artisan boost:install
```

porque la documentación oficial lo presenta como una dependencia de desarrollo instalable después. [laravel.com](#) +1

Selecciona **No** y continúa. Luego me muestras la siguiente pantalla o el resultado final del comando `laravel new`.



```
Using version ^4.1 for pestphp/pest-plugin-laravel
./composer.json has been updated
Do not run Composer as root/super user! See https://getcomposer.org/root for details
Continue as root/super user [yes]?
```

A que se refiere esta ultima opcion?

Pensado durante 11s >

Esa última opción significa esto:

Composer detectó que lo estás ejecutando como root / superusuario, y te pregunta si quieres continuar de todos modos.

El mensaje:

```
Do not run Composer as root/super user
Continue as root/super user [yes]?
```

quiere decir:

- yes → seguir instalando como administrador
- no → cancelar

Qué debes hacer en tu caso

Te recomiendo:

no continuar como root.

Es decir, responde:

</> Bash

n



o simplemente presiona la opción para **No**, si te la permite.

Por qué no conviene seguir como root

Porque después te puede dejar problemas de permisos en el proyecto, por ejemplo:

- archivos del proyecto creados con dueño `root`
- errores al usar `composer install`
- errores con `npm install`
- problemas al ejecutar `php artisan`
- dificultad para editar o borrar archivos desde tu usuario normal

En desarrollo local con Laravel, lo correcto es trabajar con tu **usuario normal de Ubuntu/WSL**, no con `sudo` ni como `root` .

Qué debes revisar ahora

Primero verifica con qué usuario estás en WSL:

```
</> Bash
```

```
whoami
```



Si te responde algo como:

```
</> Bash
```

```
root
```



entonces estás dentro como superusuario, y por eso salió ese aviso.

Si te responde algo como:

```
</> Bash
```

```
jesus
```



o cualquier nombre de usuario normal, entonces probablemente ejecutaste el comando usando `sudo` .

Qué hacer según el caso

Caso 1: abriste WSL como `root`

Sal de esa sesión y entra con tu usuario normal.

Puedes probar:

```
</> Bash
```



```
exit
```

o abrir otra terminal WSL con tu usuario habitual.

Luego verifica otra vez:

```
</> Bash
```



```
whoami
```

Caso 2: ejecutaste `laravel new` con `sudo`

Entonces no uses `sudo`.

Hazlo otra vez así, sin `sudo`:

```
</> Bash
```



```
cd ~/proyectos  
laravel new agenda-estudiantil
```

Si ya se alcanzaron a crear archivos como root

Puede pasar que ya se haya creado parte del proyecto con permisos incorrectos.

Entonces corrige el propietario con:

```
</> Bash
```



```
sudo chown -R $USER:$USER ~/proyectos/agenda-estudiantil
```

Luego revisa:

```
</> Bash
```



```
ls -la ~/proyectos/agenda-estudiantil
```

y vuelve a entrar al proyecto con tu usuario normal.

Mi recomendación exacta ahora

Haz esto en orden:

</> Bash



```
whoami  
pwd
```

Si `whoami` dice `root`, no sigas con ese instalador.

Cáncélalo, entra con tu usuario normal y vuelve a ejecutar el comando.

La decisión correcta en esa pantalla es:

No continuar como root.

Pégame la salida de:

</> Bash



```
whoami  
pwd
```

y te digo exactamente cómo corregirlo.



```
INFO No publishable resources for tag [laravel-assets].  
No security vulnerability advisories found.  
Would you like to run npm install and npm run build? ☐  
● Yes / ○ No
```

Para mi caso, cual seria la mejor opcion a escoger?

Pensado durante 17s >

Para tu caso, la mejor opción es:

Yes

Porque esa opción le dice al instalador que ejecute automáticamente:

</> Bash



```
npm install  
npm run build
```

Eso sirve para **instalar las dependencias frontend del proyecto** y luego **compilar los assets** del starter kit, que en Laravel usan **Vite**. La documentación oficial indica que los starter kits ya traen la configuración correcta de Laravel + frontend + Vite, y que los assets se procesan con `npm run build`; también señala que para desarrollar con Laravel necesitas **Node y NPM o Bun** para compilar esos assets. Lara... +2

En tu caso concreto, además, tú ya elegiste un starter kit con frontend y autenticación, así que **sí conviene dejar que lo haga de una vez**. Eso te evita tener que ejecutar manualmente después los mismos comandos dentro del proyecto. Lara... +1

Solo escogería **No** si te ocurre alguna de estas situaciones: no tienes `npm`, no tienes conexión para descargar paquetes, o quieres posponer la instalación de dependencias para revisarlas luego. En ese caso, después tendrías que correrlo manualmente dentro de la carpeta del proyecto. Laravel también documenta el uso de `composer run dev` para arrancar el entorno local una vez que las dependencias están listas. Lara... +1

Entonces aquí selecciona:

Yes

Si después de eso termina la creación del proyecto, el siguiente paso es revisar que se haya creado bien y entrar a la carpeta para levantarlo.

     ...  Fuentes

```
added 92 packages, and audited 93 packages in 10s

26 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

> build
> vite build

vite v7.3.1 building client environment for production...
✓ 2 modules transformed.
Generated an empty chunk: "app".
public/build/manifest.json 0.33 kB | gzip: 0.17 kB
public/build/assets/app-STt_G2Zz.css 229.61 kB | gzip: 30.90 kB
public/build/assets/app-l0sNRNKZ.js 0.00 kB | gzip: 0.02 kB
✓ built in 324ms
[INFO] Application ready in [agenda-estudiantil]. You can start your local development using:
→ cd agenda-estudiantil
→ composer run dev

New to Laravel? Check out our documentation. Build something amazing!

root@Jesus-Rojas:/home/Projects/agenda-estudiantil#
```

Esta es la respuesta que recibo al finalizar la instalacion

Pensado durante 21s >

La instalación **sí terminó bien**.

Lo importante de esa salida es esto:

- Application ready in [agenda-estudiantil] → el proyecto fue creado.
- npm install y vite build terminaron correctamente.
- found 0 vulnerabilities → no detectó vulnerabilidades en esos paquetes.
- built in 324ms → los assets frontend se compilaron bien.

Pero hay un detalle clave en tu captura:

 Bash



```
root@Jesus-Rojas:/home/Projects/agenda-estudiantil#
```

Eso confirma que **estás dentro como root** . Ese es el punto que hay que corregir **antes de seguir**.

Qué significa entonces esta respuesta

En resumen:

- el proyecto quedó creado
- la compilación salió bien

- pero fue creado/ejecutado como root

Lo más correcto ahora

Como apenas estás empezando y todavía no has desarrollado nada dentro del proyecto, la opción más limpia es:

borrar este proyecto y volverlo a crear con tu usuario normal de WSL, no con root

Eso te evita problemas de permisos más adelante con:

- composer
- npm
- php artisan
- edición de archivos en VS Code

Cómo comprobarlo

Ejecuta esto:

```
</> Bash
```



```
whoami  
pwd
```

Si `whoami` responde `root` , entonces no debemos seguir trabajando así.

Opción recomendada: rehacerlo bien desde el inicio

1) Sal de root

Si estás en una shell root, prueba:

```
</> Bash
```



```
exit
```

Si sigue entrando como root, abre una nueva terminal de Ubuntu/WSL y verifica:

```
</> Bash
```



```
whoami
```

Debes quedar con un usuario normal, algo como:

```
</> Bash
```



```
jesus
```

o el nombre real de tu usuario.

2) Borra el proyecto creado como root

Como todavía estás en fase inicial, hazlo solo si no has metido nada manual importante:

```
</> Bash
```



```
sudo rm -rf /home/Projects/agenda-estudiantil
```

3) Crea una carpeta de trabajo dentro de tu home

Por ejemplo:

```
</> Bash
```



```
mkdir -p ~/Projects  
cd ~/Projects
```

4) Vuelve a crear el proyecto, ahora sin root ni sudo

```
</> Bash
```



```
laravel new agenda-estudiantil  
cd agenda-estudiantil  
composer run dev
```

Opción alternativa: corregir permisos sin recrearlo

Si no quieres borrarlo, puedes intentar reparar propietario y permisos.

Primero identifica tu usuario normal:


```
</> Bash
```



```
ls /home
```

Supongamos que tu usuario normal es `jesus` , entonces:

```
</> Bash
```



```
sudo chown -R jesus:jesus /home/Projects/agenda-estudiantil
```

Luego sales de root, entras con `jesus` , y pruebas:

```
</> Bash
```



```
cd /home/Projects/agenda-estudiantil  
composer run dev
```

Pero, como todavía no has avanzado nada funcional, **rehacerlo limpio** sigue siendo la mejor opción.

Sobre este mensaje

```
</> Bash
```



```
Generated an empty chunk: "app"
```

Por sí solo, **no parece un error bloqueante**.

Como el build terminó bien y Laravel marcó la app como lista, no es lo que más preocupa ahora. El problema real es haber trabajado como `root` .

Mi recomendación directa

Haz esto ahora:

```
</> Bash
```



```
whoami  
ls /home
```

y pégame la salida. Con eso te digo exactamente cuál comando usar para salir de root y recrear el proyecto correctamente.



Hay algo que debes entender, el usuario root, es el unico que existe en mi instancia de WSL2, indica el comando para comprobarlo

Pensado durante 11s

Sí. Para comprobarlo desde tu instancia WSL2, usa este comando:

 Bash



```
getent passwd | cut -d: -f1
```

Ese comando te lista **todos los usuarios del sistema** que existen en esa distro WSL.

Si quieres verificar específicamente si existe algún **usuario humano normal** además de root , usa este:

 Bash



```
awk -F: '$3 >= 1000 {print $1, $3, $6}' /etc/passwd
```

Cómo interpretarlo:

- Si ahí **solo aparece** root , entonces efectivamente estás usando una instancia muy mínima o atípica.
- Si aparecen otros nombres además de root , entonces sí existen otros usuarios locales en esa distro.

Ahora bien, en Linux root normalmente tiene UID 0 , así que para ver de forma clara los usuarios más relevantes puedes usar:

 Bash



```
awk -F: '{print "usuario=" $1, "uid=" $3, "home=" $6}' /etc/passwd
```

Y para confirmar con qué usuario estás actualmente:

</> Bash



```
whoami  
id
```

Si quieres una comprobación más enfocada a tu caso, ejecuta estos tres:

</> Bash



```
whoami  
getent passwd | cut -d: -f1  
awk -F: '$3 >= 1000 || $1=="root" {print $1, $3, $6}' /etc/passwd
```

Pégame la salida y te digo si realmente tu WSL2 quedó creado solo con `root` o si hay otro usuario que no estás viendo.



Este comando: "getent passwd | cut -d: -f1", responde con:

```
"root  
daemon  
bin  
sys  
sync  
games  
man  
lp  
mail  
news  
uucp  
proxy  
www-data  
backup  
list  
irc  
_apt  
nobody  
systemd-network  
systemd-timesync  
dhcpcd
```

```

messagebus
syslog
systemd-resolve
uuidd
landscape
polkitd
redis", y este: "awk -F: '$3 >= 1000 {print $1, $3, $6}'
/etc/passwd", responde con: "nobody 65534 /nonexistent",
por eso debo quedarme con el usuario root

```

Pensado durante 1m 4s >

Tu salida **sí confirma** que en esa distro WSL no tienes un usuario humano normal creado todavía: solo aparecen cuentas del sistema y `root`, y el hecho de que no salga ningún usuario con UID típico de usuario normal encaja con una distro/importación que arranca directamente como `root`. Microsoft además indica que, cuando una distro se importa con `--import`, **por defecto inicia como** `root`, y que luego puedes configurar tu propio usuario por defecto. [Microsoft Learn](#) +1

Pero la conclusión correcta **no es** "entonces debo quedarme con `root`".

La conclusión correcta es: **ahora mismo solo tienes root, así que debes crear un usuario normal y ponerlo como usuario por defecto**. En Ubuntu, `adduser <nombre>` crea un usuario normal, y `adduser <usuario> <grupo>` añade un usuario existente a un grupo existente. Microsoft también documenta que puedes establecer el usuario por defecto en WSL mediante `ws1.conf`. [manpages.ubu...](#) +3

Hazlo así, desde tu sesión actual como `root`:

</> Bash



```

adduser jesus
adduser jesus sudo

```

El primer comando te va a pedir contraseña y datos básicos del usuario. El segundo lo agrega al grupo `sudo`, para que puedas administrar el sistema sin trabajar permanentemente como `root`. En Ubuntu, `passwd` es la herramienta estándar para gestionar contraseñas, y `adduser` es la forma recomendada de crear un usuario normal.

[manpages.ubu...](#) +2

Luego configura ese usuario como predeterminado en WSL editando `/etc/wsl.conf` :

</> Bash



```
cat > /etc/wsl.conf <<'EOF'  
[user]  
default=jesus  
EOF
```

Microsoft indica que `wsl.conf` permite fijar `user.default=<nombre>` como usuario por defecto de la distro. [Microsoft Learn +1](#)

Después, desde **PowerShell o CMD en Windows**, ejecuta:

</> PowerShell



```
wsl --shutdown
```

y vuelve a abrir tu distro WSL. Al entrar de nuevo, comprueba:

</> Bash



```
whoami  
id
```

Si todo quedó bien, ya no debería entrar como `root` , sino como `jesus` . Microsoft documenta que, tras definir el usuario por defecto, WSL usa ese usuario al iniciar la distribución. [Microsoft Learn +1](#)

Como ya creaste el proyecto Laravel siendo `root` , corrige también la propiedad de esa carpeta para que el nuevo usuario pueda trabajar sin problemas:

</> Bash



```
chown -R jesus:jesus /home/Projects/agenda-estudiantil
```

Eso te evita errores futuros con `composer` , `npm` , `php artisan` y edición de archivos. La necesidad de corregir propiedad aquí se desprende de que los archivos fueron creados por `root` , mientras que el usuario nuevo será quien los use de ahora en adelante. [Microsoft Learn +1](#)

En resumen: **sí, hoy solo tienes `root` ; no, no debes quedarte desarrollando como `root` .**
Lo correcto es crear tu usuario normal, hacerlo predeterminado y luego continuar el

proyecto con ese usuario. Si quieres, el siguiente paso te lo dejo ya armado para tu caso exacto con los comandos completos usando el nombre de usuario que prefieras.

       Fuentes

Tengo otros proyectos, que trabajan en base a este usuario, hacer esto seria descontrolar mi flujo de trabajo, prefiero quedarme con mi estructura de usuario actual, por favor

Pensado durante 24s >

Entendido. En ese caso, **nos quedamos con tu flujo actual usando** `root` y trabajamos de forma consistente sobre esa base.

No es la práctica más limpia, pero **sí se puede desarrollar así** si mantienes disciplina en el entorno. Lo importante es **no mezclar usuarios** ni empezar luego a crear archivos con otro usuario distinto, porque ahí sí aparecen los problemas de permisos.

Entonces, cómo seguimos en tu caso

Tu regla a partir de ahora será esta:

- trabajar siempre con el mismo usuario `root`
- no alternar entre `root` y otro usuario dentro de ese proyecto
- no usar `sudo` innecesariamente, porque ya estás como `root`

Qué hacer con el aviso de Composer al usar root

Como decidiste mantener esa estructura, cuando Composer vuelva a advertirte por ejecutarse como superusuario, puedes permitirlo.

Para no ver ese aviso a cada rato en esta sesión:

 Bash

```
export COMPOSER_ALLOW_SUPERUSER=1
```



Y si quieres dejarlo persistente para futuras terminales:

</> Bash



```
echo 'export COMPOSER_ALLOW_SUPERUSER=1' >> ~/.bashrc  
source ~/.bashrc
```

En qué estado estás ahora mismo

Según tu captura anterior, el proyecto ya quedó creado correctamente en:

</> Bash



```
/home/Projects/agenda-estudiantil
```

Así que ahora el siguiente paso real no es recrear nada, sino **entrar al proyecto y levantarlo**.

Siguiente paso exacto

Ejecuta esto:

</> Bash



```
cd /home/Projects/agenda-estudiantil  
php artisan migrate  
composer run dev
```

Qué hace cada comando

```
php artisan migrate
```

crea la estructura inicial de la base de datos de Laravel.

```
composer run dev
```

levanta el entorno de desarrollo que el instalador te sugirió al final.

Antes de correr migrate , verifica SQLite

Como elegimos una ruta simple con SQLite, verifica que exista el archivo de base de datos o créalo si falta.

Primero revisa tu .env :

</> Bash



```
grep DB_CONNECTION .env
```

```
grep DB_DATABASE .env
```

Si ves que usa SQLite, entonces asegúrate de que exista el archivo:

```
</> Bash

mkdir -p database
touch database/database.sqlite
```

Luego:

```
</> Bash

php artisan migrate
```

Si `php artisan migrate` falla

El error más común aquí sería que falte el archivo SQLite o que el `.env` no esté apuntando bien.

En ese caso revisa esto:

```
</> Bash

cat .env | grep DB_
ls -l database
```

Y confirma que tengas algo parecido a:

```
</> env

DB_CONNECTION=sqlite
DB_DATABASE=/home/Projects/agenda-estudiantil/database/database.sqlite
```

o, según cómo lo haya dejado el instalador, una ruta relativa válida.

Después de levantar el proyecto

Cuando `composer run dev` quede corriendo, Laravel normalmente te mostrará la URL local en consola. Ahí podrás abrir la aplicación y comprobar que:

- la página inicial carga
- el login está disponible
- el proyecto quedó funcional

Lo siguiente después de validar que abre

Una vez que confirme que la app carga localmente, el paso correcto será:

1. instalar **Filament**
2. crear el usuario admin de Filament
3. diseñar las primeras tablas del proyecto:
 - secciones
 - materias
 - estudiantes
 - docentes
 - horario
 - tareas

Mi recomendación práctica contigo

Como vas a seguir con `root` , mantengamos estas reglas simples:

 **Bash**



```
cd /home/Projects/agenda-estudiantil
export COMPOSER_ALLOW_SUPERUSER=1
touch database/database.sqlite
php artisan migrate
composer run dev
```

Haz eso y pégame la salida de:

 **Bash**



```
php artisan migrate
```

para indicarte el siguiente paso exacto sin romper tu flujo actual.



Esto se puede hacer mejor en visual studio: "cd
/home/Projects/agenda-estudiantil
php artisan migrate
composer run dev"?

Pensado durante 19s >

Sí. En tu caso, **es mejor hacerlo desde VS Code conectado a WSL**, no desde una consola aparte.

La forma más cómoda es esta:

1. En tu terminal WSL, entra al proyecto y abre la carpeta en VS Code:

</> Bash



```
cd /home/Projects/agenda-estudiantil  
code .
```

2. Ya dentro de VS Code, abre la terminal integrada con `Ctrl + ñ` o desde `Terminal > New Terminal`.
3. En esa terminal, ejecuta:

</> Bash



```
export COMPOSER_ALLOW_SUPERUSER=1  
touch database/database.sqlite  
php artisan migrate  
composer run dev
```

Por qué así es mejor

Porque te queda todo en un solo lugar:

- editor de código
- terminal WSL
- logs de Laravel
- navegación de archivos del proyecto

y además VS Code abierto con `code .` sobre WSL trabaja directamente contra `/home/Projects/agenda-estudiantil`, que es justo lo correcto en tu flujo.

Cómo usarlo bien dentro de VS Code

Hazlo con **dos terminales**:

Terminal 1

Para comandos puntuales:

</> Bash



```
php artisan migrate
```

Terminal 2

Para dejar corriendo el entorno:

</> Bash



```
composer run dev
```

Así no bloqueas la única terminal con el proceso de desarrollo.

Orden correcto ahora mismo

Dentro de VS Code:

</> Bash



```
cd /home/Projects/agenda-estudiantil
export COMPOSER_ALLOW_SUPERUSER=1
touch database/database.sqlite
php artisan migrate
composer run dev
```

Qué deberías ver después

- `php artisan migrate` debe terminar sin error
- `composer run dev` debe dejar el servidor levantado
- Laravel te mostrará una URL local, normalmente algo tipo `http://127.0.0.1:8000`

Recomendación práctica extra

Como trabajas siempre con `root`, te conviene dejar esta variable fija en tu shell para no repetirla:

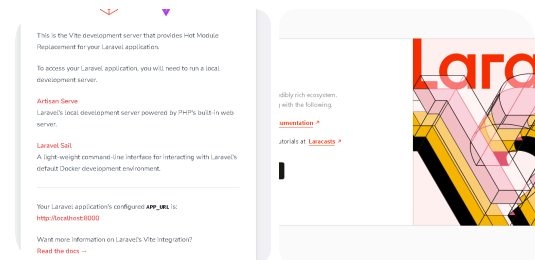
</> Bash



```
echo 'export COMPOSER_ALLOW_SUPERUSER=1' >> ~/.bashrc
source ~/.bashrc
```

Entonces ya no tendrás que escribirla en cada terminal.

Hazlo desde VS Code y pégame la salida de `php artisan migrate`.



Ok, al levantar el proyecto, esta es la vista (primer capture de pantalla) de: "http://localhost:5173/", y esta es la vista (segundo capture de pantalla), es la respuesta de: "http://localhost:8000/", y esta es la respuesta de la consola:

```
" INFO Nothing to migrate.
```

```
> Composer\Config::disableProcessTimeout
> npx concurrently -c "#93c5fd,#c4b5fd,#fb7185,#fdb74"
"php artisan serve" "php artisan queue:listen --tries=1 --
timeout=0" "php artisan pail --timeout=0" "npm run dev" --
names=server,queue,logs,vite --kill-others
[vite]
[vite] > dev
[vite] > vite
[vite]
[queue]
[queue] INFO Processing jobs from the [default] queue.
[queue]
[logs]
[logs] INFO Tailing application logs. Press
Ctrl+C to exit
[logs] Use -v|-vv to show more
details
[vite]
[vite] VITE v7.3.1 ready in 434 ms
[vite]
[vite] → Local: http://localhost:5173/
```

```
[vite] → Network: use --host to expose
[vite]
[vite] LARAVEL v12.53.0 plugin v2.1.0
[vite]
[vite] → APP_URL: http://localhost:8000
[server]
[server] INFO Server running on [http://127.0.0.1:8000].
[server]
[server] Press Ctrl+C to stop the server
[server] "
```

Pensado durante 14s >

Vas bien. **Ese comportamiento es correcto.**

Lo que estás viendo significa esto:

`http://localhost:8000/`

Esa es **tu aplicación Laravel real**.

Por eso allí ves la página de bienvenida de Laravel.

`http://localhost:5173/`

Ese **no es el sitio principal**.

Ese puerto corresponde al **servidor de desarrollo de Vite**, que Laravel usa para servir y recargar los assets frontend en caliente mientras desarrollas.

En otras palabras:

- **8000** = backend + aplicación Laravel
- **5173** = frontend dev server de Vite

Cómo interpretar tu consola

Tu salida indica que todo quedó levantado correctamente:

- `Nothing to migrate.`

Ya no había migraciones pendientes.

- `php artisan serve`
Levantó Laravel en `http://127.0.0.1:8000`
- `php artisan queue:listen`
Dejó el worker de colas activo
- `php artisan pail`
Está mostrando logs en tiempo real
- `npm run dev`
Levantó Vite en `http://localhost:5173`

Así que tu entorno local ya está funcionando bien.

Qué URL debes usar tú normalmente

Usa esta:

<> Plain text



`http://localhost:8000`

No uses `5173` como URL principal para navegar la app.

Ese puerto solo lo usa Vite para los assets mientras el servidor de desarrollo está encendido.

Entonces, ¿hay algún problema?

No.

Lo que ves en ambos puertos es exactamente lo esperable en un proyecto Laravel moderno con Vite.

Qué sigue ahora

Ya que el proyecto base está funcionando, el siguiente paso correcto es:

1. Abrir una nueva terminal en VS Code

Deja corriendo `composer run dev` en la terminal actual.

2. En la nueva terminal, entra al proyecto

<> Bash



```
cd /home/Projects/agenda-estudiantil
export COMPOSER_ALLOW_SUPERUSER=1
```

3. Instalar Filament

</> Bash



```
composer require filament/filament:"^5.0"  
php artisan filament:install --panels  
php artisan make:filament-user
```

Después de eso, normalmente podrás entrar al panel en:

</> Plain text



<http://localhost:8000/admin>

Punto importante en tu caso

Como elegiste **React** al crear el proyecto, más adelante tendremos que decidir una de estas dos rutas:

- usar **Filament solo para admin/docentes** y React para el portal estudiantil
- o mantener todo el portal principal con React/Inertia y separar claramente el panel admin

Para tu proyecto, eso está bien y se puede manejar sin problema.

La conclusión aquí es:

tu entorno local ya quedó correctamente levantado y listo para el siguiente paso.

Ejecuta ahora la instalación de Filament en una terminal nueva.



↳ Punto importante en tu caso ----- Como elegiste **React** al crear el proyecto, más adelante tendremos que decidir una de estas dos rutas: * usar **Filament solo para admin/docentes** y React para el portal estudiantil * o mantener todo el portal principal con...

en esta parte, elimine el proyecto y lo instale con: "Livewire", de todas formas, dame el comando para comprobarlo, por favor

Pensado durante 5s

Sí. La forma más directa de comprobar si tu proyecto quedó creado con **Livewire** es revisar las dependencias del proyecto.

Dentro de la carpeta del proyecto, ejecuta:

```
<> Bash
cat composer.json | grep livewire
```



Si está con Livewire, deberías ver algo parecido a esto:

```
<> Bash
"livewire/livewire": "^3.x"
```



También puedes usar una comprobación más clara:

```
<> Bash
composer show livewire/livewire
```



Si ese paquete está instalado, Composer te mostrará la información del paquete.

Si no está instalado, te dirá que no existe en el proyecto.

Otra forma útil es revisar la estructura típica del starter kit con Livewire:

```
<> Bash
ls resources/views
ls app/Livewire
```



Y también puedes comprobar si existe el layout típico de Livewire/Blade en vez de una estructura React/Inertia.

Para contrastar, revisa si **no** está Inertia/React:

```
<> Bash
composer show inertiajs/inertia-laravel
npm list @inertiajs/react
```



Mi recomendación

Usa estos dos primero, que son los más fiables:

 Bash 

```
cd /home/Projects/agenda-estudiantil
composer show livewire/livewire
composer show inertiajs/inertia-laravel
```

Interpretación:

- si `livewire/livewire` aparece instalado, vas por Livewire
- si `inertiajs/inertia-laravel` no aparece, entonces no estás en el starter kit React/Inertia

El comando más corto y práctico sería este:

 Bash 

```
composer show livewire/livewire
```

     ...

Esta: " "name": "laravel/livewire-starter-kit",
"livewire/blaze": "^1.0",
"livewire/flux": "^2.9.0",
"livewire/livewire": "^4.0"" es la respuesta con el
primer comando: "cat composer.json | grep livewire"

Pensado durante 5s

Sí, eso **confirma claramente** que el proyecto fue creado con **Livewire**.

Lo que te está diciendo esa salida es:

- "name": "laravel/livewire-starter-kit"
confirma que el proyecto usa el **starter kit oficial de Livewire**.
- "livewire/livewire": "^4.0"
confirma que **Livewire** está instalado como parte del proyecto.

- "livewire/flux" y "livewire/blade"
son paquetes del ecosistema Livewire que vienen integrados en ese starter kit.

Así que en este punto ya puedes dar por válido que:

tu proyecto quedó montado con Livewire, no con React/Inertia.

Conclusión práctica

Vas ahora por el camino correcto para lo que te recomendé antes:

- **Laravel + Livewire** para la app
- luego **Filament** para el panel admin/docentes
- y después construir:
 - secciones
 - materias
 - horario
 - tareas

Si quieres una segunda comprobación

Puedes ejecutar este comando también:

</> Bash



```
composer show livewire/livewire
```

Si responde con información del paquete, queda doblemente confirmado.

Siguiente paso correcto

Ahora sí, el siguiente paso natural es instalar **Filament** dentro de este proyecto:

</> Bash



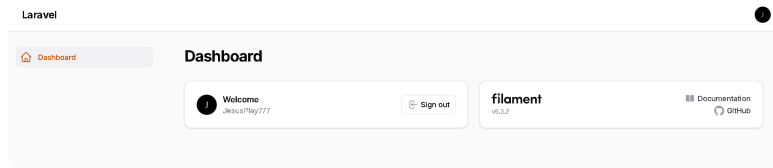
```
cd /home/Projects/agenda-estudiantil  
export COMPOSER_ALLOW_SUPERUSER=1  
composer require filament/filament:"^5.0"  
php artisan filament:install --panels  
php artisan make:filament-user
```

Después vas a poder entrar al panel en:

</> Plain text

`http://localhost:8000/admin`

Si quieres, en el siguiente paso te guío con la instalación de Filament y te explico cada pantalla o pregunta que te salga.



Ok, esto, ya lo hice: "Siguiente paso correcto

Ahora sí, el siguiente paso natural es instalar Filament dentro de este proyecto:

```
cd /home/Projects/agenda-estudiantil
export COMPOSER_ALLOW_SUPERUSER=1
composer require filament/filament:"^5.0"
php artisan filament:install --panels
php artisan make:filament-user
```

Después vas a poder entrar al panel en:

`http://localhost:8000/admin`

Si quieres, en el siguiente paso te guío con la instalación de Filament y te explico cada pantalla o pregunta que te salga.", el capture de pantalla es la prueba, voy en este punto: "y después construir:

secciones

materias

horario

tareas", puedes guiarme de forma mas detallada en este punto?

Pensado durante 2m 1s >

Sí. Desde ese punto, lo correcto es **construir primero la base de datos y las relaciones del dominio**, y después montar el CRUD en Filament encima de esa base. Laravel recomienda modelar la estructura con **migraciones y modelos Eloquent**; las migraciones funcionan como "version control" del esquema, y las relaciones Eloquent se definen como métodos en los modelos. Filament 5 además permite generar recursos CRUD con `php artisan make:filament-resource ... --generate`, leyendo las columnas ya creadas en tu base de datos. Laravel +2

Te recomiendo hacerlo en este orden:

1. usuarios con rol
2. secciones
3. materias
4. asignaciones docente-sección-materia
5. horario
6. tareas

La clave es esta: **no guardes horario y tareas "sueltos"**. Mejor cuélgalos de una entidad intermedia que diga: **"este docente da esta materia a esta sección"**. Eso te simplifica muchísimo permisos, consultas y formularios.

Fase 1: crear las entidades base

Dentro del proyecto, crea esto:

 **Bash**



```
cd /home/Projects/agenda-estudiantil
export COMPOSER_ALLOW_SUPERUSER=1

php artisan make:migration add_role_to_users_table --table=users
```

```
php artisan make:model AcademicSection -m
php artisan make:model Subject -m
php artisan make:model StudentProfile -m
php artisan make:model TeacherProfile -m
php artisan make:model TeachingAssignment -m
php artisan make:model Schedule -m
php artisan make:model Assignment -m
```

Laravel soporta `make:model ... -m` para generar el modelo y su migración al mismo tiempo. Laravel

Fase 2: define el diseño correcto de tablas

1) migración para `users.role`

Abre la migración nueva `add_role_to_users_table` y deja algo así:

<> PHP



```
<?php
```

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('role')->default('student')->after('email');
        });
    }

    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('role');
        });
    }
};
```

Roles que usaremos por ahora:

- admin

- teacher
- student

2) academic_sections

Ejemplo: 5to Año A

</> PHP



<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('academic_sections', function (Blueprint $table) {
            $table->id();
            $table->string('name'); // Ej: 5to Año A
            $table->string('school_year'); // Ej: 2026-2027
            $table->boolean('is_active')->default(true);
            $table->timestamps();

            $table->unique(['name', 'school_year']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('academic_sections');
    }
};
```

3) subjects

Ejemplo: Matemáticas, Inglés, Castellano

</> PHP



```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('subjects', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('code')->nullable();
            $table->boolean('is_active')->default(true);
            $table->timestamps();

            $table->unique('name');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('subjects');
    }
};
```

4) student_profiles

Cada estudiante pertenece a una sección.

</> PHP



```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('student_profiles', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->cascadeOnDelete();
```

```

        $table->foreignId('academic_section_id')->constrained()->cascadeOnDelete();
        $table->string('phone')->nullable();
        $table->timestamps();

        $table->unique('user_id');
    });
}

public function down(): void
{
    Schema::dropIfExists('student_profiles');
}
};

```

5) teacher_profiles

</> PHP



<?php

```

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('teacher_profiles', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->cascadeOnDelete();
            $table->string('phone')->nullable();
            $table->timestamps();

            $table->unique('user_id');
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('teacher_profiles');
    }
};

```


6) teaching_assignments

Esta es la tabla más importante.

Representa: **un docente dicta una materia a una sección.**

</> PHP



<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('teaching_assignments', function (Blueprint $table) {
            $table->id();
            $table->foreignId('academic_section_id')->constrained()->cascadeOnDelete();
            $table->foreignId('subject_id')->constrained()->cascadeOnDelete();
            $table->foreignId('teacher_id')->constrained('users')->cascadeOnDelete();
            $table->boolean('is_active')->default(true);
            $table->timestamps();

            $table->unique(['academic_section_id', 'subject_id', 'teacher_id']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('teaching_assignments');
    }
};
```

7) schedules

Cada bloque horario apunta a una asignación docente-sección-materia.

</> PHP



<?php

```

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('schedules', function (Blueprint $table) {
            $table->id();
            $table->foreignId('teaching_assignment_id')->constrained()->cascade();
            $table->string('weekday'); // Lunes, martes, etc.
            $table->time('start_time');
            $table->time('end_time');
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('schedules');
    }
};

```

8) assignments

Cada tarea también apunta a una asignación docente-sección-materia.

</> PHP



<?php

```

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('assignments', function (Blueprint $table) {
            $table->id();
            $table->foreignId('teaching_assignment_id')->constrained()->cascade();

```

```

        $table->string('title');
        $table->text('description');
        $table->date('due_date')->nullable();
        $table->timestamp('published_at')->nullable();
        $table->boolean('is_active')->default(true);
        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('assignments');
}
};

```

Fase 3: relaciones en los modelos

Laravel usa métodos en los modelos para definir relaciones Eloquent. Laravel

app/Models/User.php

Agrega algo así:

</> PHP



```
<?php
```

```

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasOne;
use Illuminate\Database\Eloquent\Relations\HasMany;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    use Notifiable;

    public const ROLE_ADMIN = 'admin';
    public const ROLE_TEACHER = 'teacher';
    public const ROLE_STUDENT = 'student';

    protected $fillable = [
        'name',
    ];
}

```

```
        'email',
        'password',
        'role',
    ];

    protected $hidden = [
        'password',
        'remember_token',
    ];

    public function studentProfile(): HasOne
    {
        return $this->hasOne(StudentProfile::class);
    }

    public function teacherProfile(): HasOne
    {
        return $this->hasOne(TeacherProfile::class);
    }

    public function teachingAssignments(): HasMany
    {
        return $this->hasMany(TeachingAssignment::class, 'teacher_id');
    }

    public function isAdmin(): bool
    {
        return $this->role === self::ROLE_ADMIN;
    }

    public function isTeacher(): bool
    {
        return $this->role === self::ROLE_TEACHER;
    }

    public function isStudent(): bool
    {
        return $this->role === self::ROLE_STUDENT;
    }
}
```

app/Models/AcademicSection.php

</> PHP



```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

class AcademicSection extends Model
{
    protected $fillable = [
        'name',
        'school_year',
        'is_active',
    ];

    public function studentProfiles(): HasMany
    {
        return $this->hasMany(StudentProfile::class);
    }

    public function teachingAssignments(): HasMany
    {
        return $this->hasMany(TeachingAssignment::class);
    }
}
```

app/Models/Subject.php

</> PHP



```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Subject extends Model
{
    protected $fillable = [
        'name',
        'code',
        'is_active',
    ];
}
```

```
public function teachingAssignments(): HasMany
{
    return $this->hasMany(TeachingAssignment::class);
}
```

app/Models/StudentProfile.php

</> PHP



<?php

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class StudentProfile extends Model
{
    protected $fillable = [
        'user_id',
        'academic_section_id',
        'phone',
    ];

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }

    public function academicSection(): BelongsTo
    {
        return $this->belongsTo(AcademicSection::class);
    }
}
```

app/Models/TeacherProfile.php

</> PHP



<?php

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class TeacherProfile extends Model
{
    protected $fillable = [
        'user_id',
        'phone',
    ];

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}
```

app/Models/TeachingAssignment.php

</> PHP



<?php

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\HasMany;

class TeachingAssignment extends Model
{
    protected $fillable = [
        'academic_section_id',
        'subject_id',
        'teacher_id',
        'is_active',
    ];

    public function academicSection(): BelongsTo
    {
        return $this->belongsTo(AcademicSection::class);
    }

    public function subject(): BelongsTo
```

```
{
    return $this->belongsTo(Subject::class);
}

public function teacher(): BelongsTo
{
    return $this->belongsTo(User::class, 'teacher_id');
}

public function schedules(): HasMany
{
    return $this->hasMany(Schedule::class);
}

public function assignments(): HasMany
{
    return $this->hasMany(Assignment::class);
}
}
```

app/Models/Schedule.php

</> PHP



<?php

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Schedule extends Model
{
    protected $fillable = [
        'teaching_assignment_id',
        'weekday',
        'start_time',
        'end_time',
    ];

    public function teachingAssignment(): BelongsTo
    {
        return $this->belongsTo(TeachingAssignment::class);
    }
}
```


app/Models/Assignment.php

</> PHP



<?php

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Assignment extends Model
{
    protected $fillable = [
        'teaching_assignment_id',
        'title',
        'description',
        'due_date',
        'published_at',
        'is_active',
    ];

    protected function casts(): array
    {
        return [
            'due_date' => 'date',
            'published_at' => 'datetime',
            'is_active' => 'boolean',
        ];
    }

    public function teachingAssignment(): BelongsTo
    {
        return $this->belongsTo(TeachingAssignment::class);
    }
}
```

Laravel soporta casts nativos en Eloquent para transformar valores al leer/escribir atributos.

Laravel

Fase 4: ejecutar migraciones

Cuando guardes todo:

</> Bash



```
php artisan migrate
```

Como ya tienes un usuario admin creado para Filament, **no uses** `migrate:fresh` porque te borraría la base actual.

Fase 5: montar el CRUD en Filament

Cuando la base ya exista, usa Filament para generar los recursos. Filament 5 permite generar recursos CRUD y, con `--generate`, levantar formulario y tabla a partir del esquema de la base de datos. Filament

Ejecuta:

</> Bash



```
php artisan make:filament-resource AcademicSection --generate
php artisan make:filament-resource Subject --generate
php artisan make:filament-resource TeachingAssignment --generate
php artisan make:filament-resource Schedule --generate
php artisan make:filament-resource Assignment --generate
```

Eso te va a dejar un CRUD base en `/admin`.

Fase 6: qué debes cargar primero en el panel

El orden correcto de uso del panel será:

1. crear docentes y marcar `role=teacher`
2. crear estudiantes y marcar `role=student`
3. crear secciones
4. crear materias
5. crear `teaching assignments`
6. crear horario
7. crear tareas

No intentes cargar horario o tareas antes de tener `teaching_assignments`, porque ambos dependen de esa relación.

Fase 7: permisos, pero sin complicarte todavía

Laravel maneja autorización con **gates** y **policies**. Más adelante vamos a necesitar policies, sobre todo para que un estudiante solo vea lo de su sección y un docente solo publique tareas en sus materias/secciones. Pero para este momento, primero cierra la estructura de datos y el CRUD. Laravel

Qué haría yo ahora, exactamente

Tu siguiente bloque de trabajo debería ser este:

</> Bash



```
cd /home/Projects/agenda-estudiantil
php artisan make:migration add_role_to_users_table --table=users
php artisan make:model AcademicSection -m
php artisan make:model Subject -m
php artisan make:model StudentProfile -m
php artisan make:model TeacherProfile -m
php artisan make:model TeachingAssignment -m
php artisan make:model Schedule -m
php artisan make:model Assignment -m
```

Después pegas el contenido de las migraciones y modelos que te dejé arriba, y luego:

</> Bash



```
php artisan migrate
php artisan make:filament-resource AcademicSection --generate
php artisan make:filament-resource Subject --generate
php artisan make:filament-resource TeachingAssignment --generate
php artisan make:filament-resource Schedule --generate
php artisan make:filament-resource Assignment --generate
```

Desde ahí ya tendrás el **esqueleto funcional del backend académico**.

El siguiente paso lógico es que te deje armado el bloque siguiente: **cómo ajustar los recursos de Filament para que los selects de docente, sección y materia queden bien relacionados y no muestres usuarios incorrectos**.

